

SVEUČILIŠTE U ZAGREBU
FAKULTET ELEKTROTEHNIKE I RAČUNARSTVA

ZAVRŠNI RAD br. 2220

**VIZUALIZACIJA TRODIMENZIJSKIH
FRAKTALNIH OBJEKATA U WEBGL2
OKRUŽENJU**

Martin Golub

Zagreb, lipanj, 2026.

ZAVRŠNI ZADATAK br. 2220

Pristupnik: **Martin Golub (0036559810)**
Studij: Elektrotehnika i informacijska tehnologija i Računarstvo
Modul: Računarstvo
Mentorica: prof. dr. sc. Željka Mihajlović

Zadatak: **Vizualizacija trodimenzijskih fraktalnih objekata u WebGL2 okruženju**

Opis zadatka:

U suvremenoj računalnoj grafici za izradu složenih objekata, kakva je većina prirodnih objekata, često se koriste fraktalne strukture. Potrebno je istražiti matematičko definiranje fraktalnih objekata, te mogućnosti WebGL2 tehnologije za njihovu vizualizaciju u stvarnom vremenu. Osmisliti, razviti i optimizirati izradu prikaza fraktala u okviru web preglednika. Implementirati pripadno korisničko sučelje koje omogućava interaktivno definiranje potrebnih parametara. Omogućiti unos i bilo koje funkcije za koju je moguć prikaz funkcije udaljenosti SDF (engl. Signed Distance Function). Provesti testiranje na nizu primjera te analizirati i ocijeniti ostvarene rezultate. Razmotriti upotrebljivost dobivenih rezultata i moguće smjernice proširenja. Izraditi odgovarajući programski proizvod koristeći programski jezik JavaScript i grafičko programsko sučelje WebGL2. Rezultate rada načiniti dostupne putem Interneta. Radu priložiti algoritme, izvorne kodove i rezultate uz potrebna objašnjenja i dokumentaciju. Citirati korištenu literaturu i navesti dobivenu pomoć.

Rok za predaju rada: 12. lipnja 2026.

Sadržaj

1. Uvod	2
2. Teorija	3
2.1. Funkcije udaljenosti	3
2.2. Fraktali	4
2.3. Algoritam iscrtavanja	5
3. Implementacija	7
3.1. Korišteni alati	7
3.2. Komponente programa	8
3.3. Postupak iscrtavanja	9
3.4. Korisničko sučelje	11
3.5. Prikaz funkcija udaljenosti	12
3.5.1. Unos funkcije	12
3.5.2. Ponuđeni fraktali	13
3.5.3. Načini prikaza	13
3.6. Web stranica i izvorni kod	13
4. Rezultati i rasprava	14
5. Zaključak	15
Literatura	16
Sažetak	17
Abstract	18

1. Uvod

[uvod]

2. Teorija

2.1. Funkcije udaljenosti

Postoji više načina na koje se može zapisati neko tijelo. U računalnoj grafici se najčešće trodimenzionalni modeli opisuju mrežom trokuta. Trokuti su praktični jer imaju jednostavnu strukturu i mogu se efikasno iscrtati. Unatoč toga, postoje nedostaci takvim pristupom. Da bi se postigla zakrivljena površina, potrebno je izgraditi površinu s velikim brojem trokuta. Time površina nikad neće biti uistinu zakrivljena, nego će se sastojati od diskretnih ravnih dijelova. Neke strukture (kao na primjer spužvaste strukture) zahtijevaju toliko veliki broj trokuta da mora se napraviti kompromis brisanjem detalja strukture radi optimizacije. Detalji se općenito dodaju modelima koristeći teksture, no u tim slučajevima stvarna geometrija još uvijek ostaje pojednostavljena.

Način prikaza tijela koje će se razmatrati u ovom radu jer matematički opis. Specifično, ono što ovaj rad koristi su funkcije udaljenosti (*eng. Signed Distance Function, SDF*). Oblik funkcije je sljedeći: $f(p)$. Gdje je p točka u prostoru, a rezultat funkcije $f(p)$ je udaljenost te točke od površine tijela koju funkcija opisuje. Dimenzija prostora u kojem se mora nalaziti ulazna točka određuje funkcija. Primjer jedne takve funkcije za opis sfere je navedena ispod u jednađbi (2.1).

$$f(p)_{\text{sfera}} = |\text{centar} - p| - \text{radius} \quad (2.1)$$

Slika 2.1. skicira vrijednosti funkcije udaljenosti pravokutnika. Crte na slici prikazuju gdje su vrijednosti funkcije jednake (mogu se zamisliti kao izohipse na topografskoj karti). Bijela crta označuje gdje je vrijednost udaljenosti nula, odnosno površina tijela. Osim vraćanja vrijednost udaljenosti, drugo važno svojstvo funkcije je predznak rezul-

tata. Vrijednost je pozitivna kada je točka izvan tijela (slika 2.1., narančasti dio), a negativna kada je točka unutar tijela (slika 2.1., plavi dio). Koristeći navedena svojstva je moguće opisati neko trodimenzionalno tijelo. Nadalje, to je i dovoljno za izgraditi prikaz tog tijela.



Slika 2.1. Ilustracija funkcije udaljenosti [1]

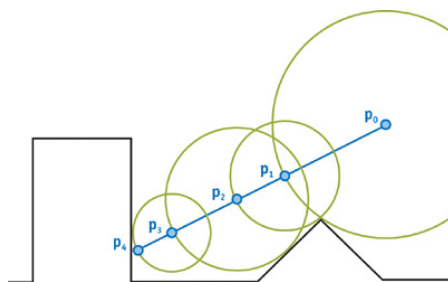
2.2. Fraktali

Najčešće se koristi matematički zapis za opis jednostavnih oblika kao ravnine, sfere ili trokuta. Ako pokušavamo matematički opisati neki kompleksniji oblik kao na primjer zec, brzo postane očito da će to biti vrlo teško. Može se pretpostaviti da je kompleksnost formule raste s kompleksnosti oblika. No, to nije uvijek slučaj. Ispada da jednostavna formula može napraviti vrlo detaljan oblik, a najbolji primjer toga su fraktali. Fraktali su objekti sa svojstvom samoponavljjanja. Odnosno, postoji uzorak (ili *pattern*) koji se ponavlja po modelu, na raznim mjestima i raznim veličinama. Ponavljanje tog uzorka može ići do beskonačnosti, što znači da je fraktal u teoriji ima beskonačno detalja. Ovdje funkcije daljine imaju koristi jer se mogu koristiti za opis fraktala.

[priroda]

2.3. Algoritam iscrtavanja

Cilj je postići prikaz fraktala. Potreban je algoritam koji uzima opis tijela u obliku funkcije u daljenosti i iscrtava trodimenzionalni prikaz tog tijela na ekranu. Koristiti će se metoda praćenja sfere, što je podvrsta metode koračenja zrake, što je još podvrsta metode praćenja zrake. Metoda praćenja zrake (*engl. Ray Tracing*) određuje boju svakog piksela pomoću zrake koja počinje u očistu kamere i prolazi kroz taj piksel. Metoda koračenja (*engl. Ray Marching*) zrake koristi definiranu zraku tako da postepeno korači u smjeru zrake tijekom računanja. Praćenje sfere (*engl. Sphere Tracing*) dalje definira koračenje tako da je duljina svakog koraka određena na temelju funkcije udaljenosti. Prikazivanjem svakog skoka i pripadajuće vrijednosti udaljenosti se dobiva slika sfera (ili kružnica ako se gleda u 2D prostoru), što se može vidjeti na slici 2.2.



Slika 2.2. Koraci zrake u praćenju sfere [2]

Ispod je naveden pseudo-kod (Algoritam 1) koji prikazuje postupak metode praćenja sfere. U svakoj iteraciji petlje se računa udaljenost pomoću funkcije udaljenosti i obavi se korak duljine te udaljenosti u smjeru zrake. Petlja završava kada je zraka dotaknula tijelo ili kada prođe granicu maksimalnog broja koraka. Mora se zadati granica koraka zbog slučaja kada zraka ne udari tijelo, nego nastavi koračiti u beskonačnost. Zbog nepreciznosti brojeva s pomičnim zarezom može doći do slučaja da zraka nikada ne dođe do udaljenosti nula od tijela. Pritom se za uvjet dodira dodaje varijabla *epsilon*, koja ima vrijednost vrlo malog broja.

Function *pracenjeSfere(org, dir)*

```
// org - ishodiste zrake
// dir - smjer zrake
// max - maksimalan broj koraka (globalna varijabla)
// epsilon - vrlo mali broj (globalna varijabla)
pos = org;
dist =  $\infty$ ;
count = 0;
while (dist > epsilon and count < max) do
    dist = sdf(pos);
    pos += dir * dist;
    count += 1;
end
return pos
```

end

Algoritam 1: Praćenje sfere

3. Implementacija

3.1. Korišteni alati

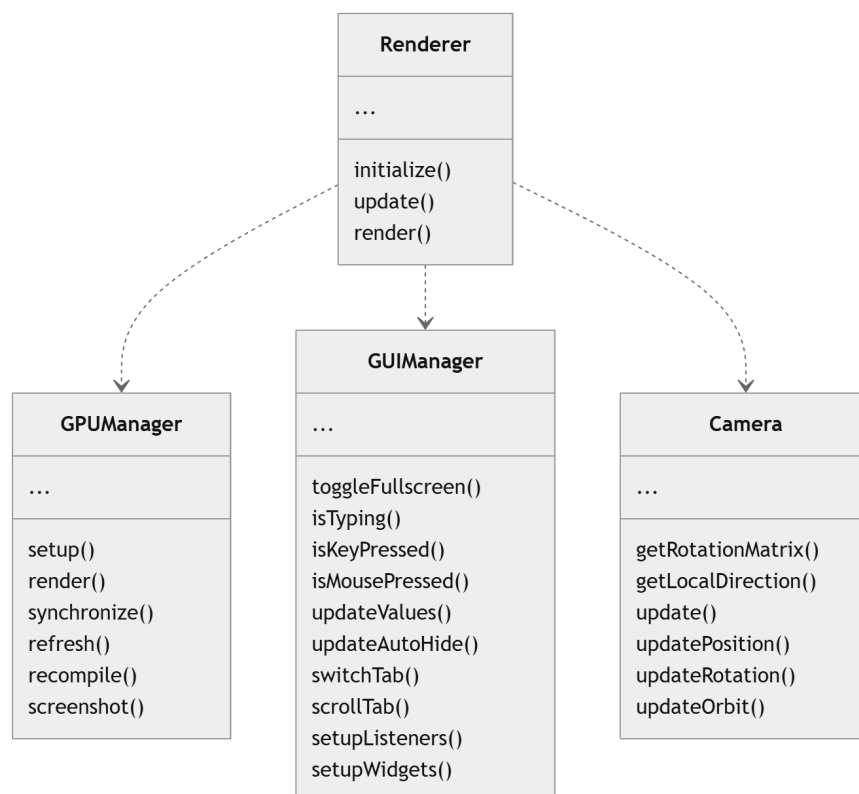
Primarni alat koji je korišten za ovaj rad je biblioteka *WebGL2*. To je varijanta popularne grafičke biblioteke OpenGL koja je prilagođena za rad na internetskom pregledniku i jeziku *JavaScript*. Mogućnost korištenja grafičkih funkcija na internetskom pregledniku ima mnoge prednosti: omogućuje kompleksne vizualizacije unutar web stranica, izradu video igara koje su dostupne bez instaliranja izvršne datoteke i generalni pristup procesorne moći grafičke kartice unutar JavaScript. *WebGL2* nije sasvim identičan OpenGL, jedna bitna razlika je što ne omogućuje izradu računskih sjenčara (*engl. compute shaders*). Takvi sjenčari su pogodni za iscrtavanje temeljenom zrakama kao praćenje sfera, no moguće je izvesti takvo iscrtavanje koristeći sjenčare fragmenta.

Kao što je već spomenuto, programski jezik koji se koristi je *JavaScript*. *JavaScript* ima svojih prednosti što se tiče grafičkih aplikacija. Zajedno s *HTML* i *CSS* pruža jednostavan i fleksibilan način izrade korisničkog sučelja za upravljanje programom. Implementacija grafičkih sučelja u drugim jezicima kao *C++* bi bila znatno teža i bilo bi potrebno koristiti datoteke kao na primjer *Dear ImGui* [3], što ima manju fleksibilnost u dizajnu. Dodatno, web aplikacije su orijentirane na portabilan dizajn na način da jedna stranica može raditi na više vrsta uređaja. To uključuje i ovaj projekt: prikaz fraktala se može otvoriti na mobitelu, a sučelje se prilagođava na orijentaciju i veličinu ekrana. Naravno, kvaliteta i brzina iscrtavanja će ovisiti o grafičkom procesoru uređaja na kojem se izvodi.

[\[github\]](#)

3.2. Komponente programa

Ovo poglavlje služi za kratak opis glavnih dijelova programa. To su četiri klase, koje se nalaze u skraćenom dijagramu razreda na slici 3.1. Klasa **Renderer** upravlja svim ostalim dijelovima: poziva inicijalizaciju scene i sučelja, učitava pohranjene postavke i upravlja komunikacijom između objekata. Zatim je klasa **GPUManager**, koja je najkompleksnija. Stvara WebGL2 kontekst, poziva WebGL2 funkcije, upravlja sjenčarima, sinkronizira rezoluciju ekrana i definira kako će se izvršiti postupak iscrtavanja okvira (detaljnije u sljedećem poglavlju). Klasa **GUIManager** implementira funkcionalnosti korisničkog sučelja tako da pri učitavanju stranice sagradi sučelje i aktivno ažurira uniformne varijable sjenčara kada se korisnik promijeni vrijednosti. Konačno je zadnja glavna klasa **Camera**. Kamera služi kao kontejner informacija potrebnih za inicijalizaciju zraka i uređuje te podatke na temelju korisničkih ulaza tipkovnicom i mišem.

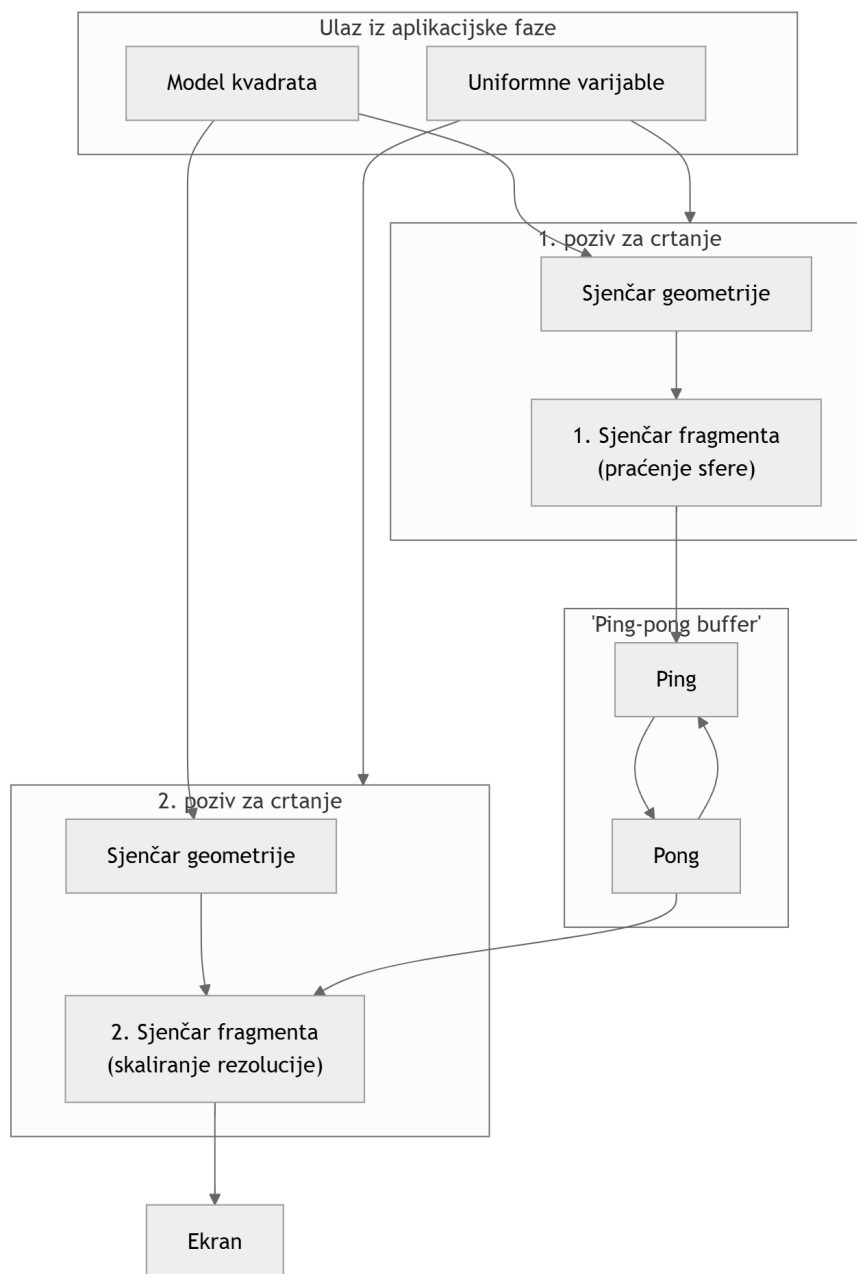


Slika 3.1. Skraćeni dijagram razreda

3.3. Postupak iscrtavanja

Iscrtavanje je stvaranje slike koja se prikaže na ekranu. Za virtualne scene s ciljem prikaza u stvarnom vremenu, frekvencija iscrtavanja mora biti dovoljno velika. Radi brzine, veliki dio postupka iscrtavanja je izveden hardverski i postoji definirani grafički protočni sustav kojeg se treba pridržavati da bi se nacrtao neki 3D model. Trodimenzionalni model prvo prolazi kroz geometrijsku fazu, a zatim rastersku fazu prije popunjavanja okvira. U WebGL2 je ta struktura obavezna, odnosno nije moguće izvršiti crtanje bez pripadajućeg sjenčara geometrije (*engl. vertex shader*) i sjenčara fragmenata (*engl. fragment shader*). U ovome slučaju, modeli koje želimo prikazati se ne sastoje od trokuta i nije potrebna geometrijska faza i bilo najbolje koristiti računske sjenčare. No, kao što je prije navedeno, WebGL2 ne omogućuje izradu računske sjenčare. Stoga je postupak iscrtavanja izveden onime što imamo.

Crtanje jednog okvira se izvodi u dva poziva na crtanje (*engl. draw calls*). U svakom pozivu na crtanje se izvodi standardna grafička protočna struktura za WebGL2: sjenčar geometrije se iterira po točkama modela i sjenčar fragmenata se iterira po fragmentima. Jedini model koji se koristi je kvadrat (dvoje trokuta) koji pokriva cijeli ekran. Taj model i geometrijski sjenčar se koriste za oba poziva za crtanje. Prvo iscrtavanje izvodi algoritam praćenje sfere, obavi sjenčanje, uzima u obzir boju prošlog okvira i rezultat se sprema u privremeni spremnik. Nije moguće iz jednog spremnika čitati i pisati u ist vrijeme, stoga je napravljeni par spremnika koji zajedno čine *ping-pong buffer*. Jedan spremnik služi za čitanje, a drugi za pisanje. Nakon svakog iscrtavanja se uloga tih spremnika zamijeni. Potrebno je imati informacije o prošlom okviru jer se koristi *anti-aliasing* metoda preko akumulacijskog spremnika. U drugom pozivu na crtanje se uzima slika u međuspremniku, podesi se boja i iscrta se u prikaznu memoriju za ekran. Prednost korištenja dvoje poziva na crtanje je to što prvi poziv može biti obavljen u manjoj rezoluciji, a drugi poziv skalira rezultat u rezoluciju koja je potrebna za ekran.

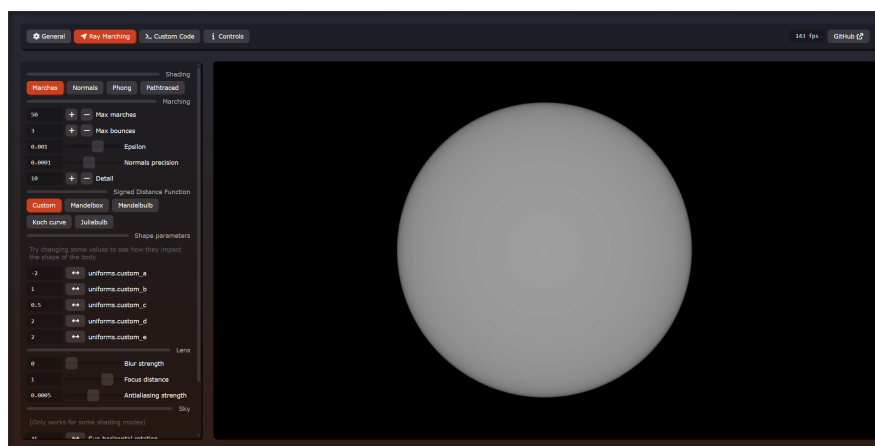


Slika 3.2. Dijagram postupka iscrtavanja

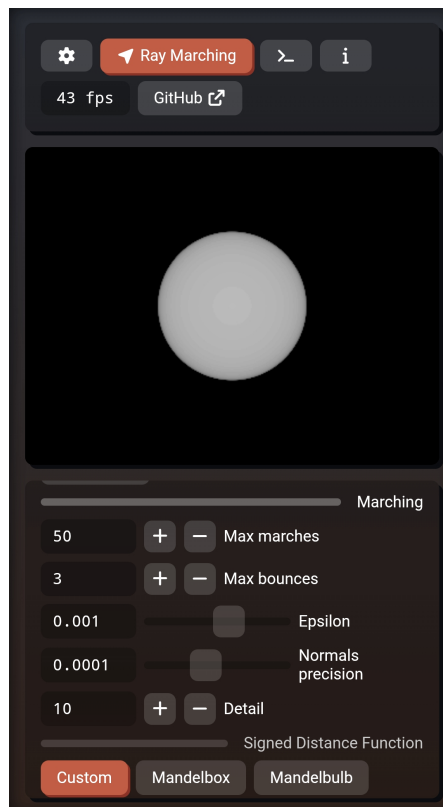
3.4. Korisničko sučelje

Dizajn korisničkog sučelja je napravljen koristeći standardnih HTML elementa i CSS pravila, a funkcionalnost sučelja je ostvarena u JavaScript. Cilj sučelja je bilo omogućiti pristup što većem broju varijabla. Uz kretanje kroz virtualni prostor, ostvarena je mogućnost podešavanja načina prikaza i čak mijenjanje funkcija daljine koje se prikazuju. Tekst sučelja je na engleskom jeziku. Upravljanje varijablama je ostvareno raznim kontrolama kao gumbi, slajderi, unos brojeva, itd. Većinu elemenata imaju kratak opis koji se pojavi ako se kratko vrijeme ostavi miš iznad njih. Slike ispod su korisničko sučelje na stolnom računalu (slika 3.3.) i na mobilnom uređaju (slika 3.4.).

Sučelje je podijeljeno u četiri kartice: generalne postavke, postavke vezane uz praćenje sfere, unos funkcije daljine i informacije o kontrolama. U kartici za informacije o kontrolama jer samo tekstualni opis gumba na tipkovnici i njihovih pripadajućih funkcionalnosti. Generalne postavke uključuju podešavanje kamere, rezolucije i slikanje ekrana. Postavke za praćenje sfere su više tehničke, imaju najveći utjecaj na brzinu iscrtavanja i mogu upravljati oblikom tijela. Konačno, u zadnjoj kartici se može unijeti funkcija daljine. Postoje kratke naputnice za izradu funkcija i jedan primjer koda.



Slika 3.3. Korisničko sučelje na stolnom računalu



Slika 3.4. Korisničko sučelje na mobilnom uređaju

3.5. Prikaz funkcija udaljenosti

3.5.1. Unos funkcije

Funkcija daljine se unosi u tekstualni okvir u obliku izvornog programskog koda jezika *GLSL*. *GLSL* je programski jezik za sjenčare kojeg koristi *WebGL2*. Funkcija daljine može koristiti druge funkcije u izvornom kodu koji se unese. Nakon unosa se treba kliknuti gumb za kompajliranje koda. Prvo se izbriše postojeći program koji *WebGL2* koristi za iscrtavanje, zatim se uneseni izvorni kod u obliku teksta spoji zajedno s izvornim kodom ostatka sjenčara fragmenta. Konačno se novi spojeni izvorni kod pošalje grafičkoj kartici, kompajlira se i stvori se novi program za iscrtavanje. Ako dođe do greške pri kompajliranju (kao na primjer sintaksna greška unesenog koda), funkcija daljine će se zamijeniti zadanom defaultnom funkcijom i ispisati će se poruka koja daje informaciju o greški radi jednostavnijeg popravljavanja koda.

Listing 3.1: Example Python Code

```
float SDF(vec3 p) {  
    float radius = uniforms.custom_b;  
    return length(p) - radius;  
}
```

[[jezik](#), [funkcije](#), [errori](#)] [[kako radi](#)]

3.5.2. Ponuđeni fraktali

[[mandelbox](#), [mandelbulb](#), [koch curve](#), [julia bulb](#)] [[slike](#)]

3.5.3. Načini prikaza

[[marches](#), [normal](#), [phong](#), [pathtracing](#)] [[slike](#)]

3.6. Web stranica i izvorni kod

Izvorni kod rada se nalazi na *GitHub* repozitoriju preko sljedeće poveznice:

<https://github.com/martzin23/webgl2-fractal-raymarcher>

Aplikacija se izvodi na web pregledniku i kompatibilna je s mobilnim uređajima. Bitno je naglasiti da program neće ispravno raditi ako uređaj ili web preglednik ne podržavaju sve WebGL2 značajke koje se koriste. Aplikacija je dostupna na navedenoj poveznici:

<https://martzin23.github.io/webgl2-fractal-raymarcher/>

4. Rezultati i rasprava

[kvaliteta] [performanse] [slike]

5. Zaključak

[zaključak]

Literatura

- [1] I. Quilez, “2d distance functions”, [pristupljeno 3.3.2026]. [Mrežno]. Adresa: <https://iquilezles.org/articles/distfunctions2d/>
- [2] W. Donnelly, “Chapter 8. per-pixel displacement mapping with distance functions”, [pristupljeno 5.5.2026]. [Mrežno]. Adresa: <https://developer.nvidia.com/gpugems/gpugems2/part-i-geometric-complexity/chapter-8-pixel-displacement-mapping-distance-functions>
- [3] Omar, “Dear imgui repository”, [pristupljeno 5.5.2026]. [Mrežno]. Adresa: <https://github.com/ocornut/imgui>

Sažetak

Vizualizacija trodimenzijskih fraktalnih objekata u WebGL2 okruženju

Martin Golub

[Unesite sažetak na hrvatskom.]

Ključne riječi: fraktali; WebGL2; koračenje zrakom; praćenje sfera; funkcija daljine

Abstract

Visualization of three-dimensional fractal objects in WebGL2

Martin Golub

[Enter the abstract in English.]

Keywords: fractals; WebGL2; ray marching; sphere tracing; signed distance function